UNIVERSIDAD POLITÉCNICA DE ATLAUTLA

MATERIA: Fundamentos de Inteligencia Artificial

DOCENTE: Mtro. Javier García Lara

ALUMNA: Denise Estrada Franco

TEMA: 2.3 BÚSQUEDA LOCAL

SEMANA 2: 19-25 octubre

GRUPO: 171

CARRERA: Tecnologías de la Información e Innovación Digital

MATRÍCULA: 24305019

FECHA DE ENTREGA: 22 de octubre de 2026

1. DATOS DEL PROBLEMA

Se trabaja con un caso real de logística de transporte: un vehículo debe visitar 18 capitales de estado de México exactamente una vez y regresar a su punto de partida, recorriendo la menor distancia total posible. Es el problema del viajante de comercio (TSP), de tipo NP-difícil.

#	CIUDAD	LATITUD	LONGITUD
0	Ciudad de México	19.43	-99.13
1	Toluca	19.29	-99.65
2	Cuernavaca	18.92	-99.23
3	Puebla	19.04	-98.20
4	Tlaxcala	19.32	-98.24
5	Pachuca	20.12	-98.73
6	Querétaro	20.59	-100.39
7	Morelia	19.70	-101.19
8	Guanajuato	21.02	-101.26

#	CIUDAD	LATITUD	LONGITUD
9	San Luis Potosi	22.15	-100.98
10	Aguascalientes	21.88	-102.29
11	Guadalajara	20.67	-103.35
12	Zacatecas	22.77	-102.58
13	Monterrey	25.67	-100.31
14	Xalapa	19.54	-96.91
15	Oaxaca	17.07	-96.72
16	Chilpancingo	17.55	-99.50
17	Villahermosa	17.99	-92.93

Tabla 1. Ciudades y coordenadas geográficas aproximadas (grados decimales)

- **Distancia entre ciudades:** fórmula de haversine (distancia sobre la esfera terrestre, radio 6,371 km) (Sinnott, 1984). Se precalcula una matriz D de 18 x 18 en kilómetros. Es distancia en línea recta, no por carretera
- **Tamaño del espacio de búsqueda:**
$$\frac{(18-1)!}{2} = \frac{17!}{2} \approx 1.78 \times 10^{14}$$
rutas distintas. Recorrerlas todas es impracticable, por eso se usa búsqueda local.
- **Referencia de calidad:** Con programación dinámica de Held y Karp (1962) se calculó el óptimo exacto (3,617.63 km, 6,4 s). Esto permite medir qué tan cerca queda cada algoritmo.
- **Las coordenadas son aproximadas (centros urbanos):** por lo que los kilómetros son una estimación para comparar algoritmos y no una distancia carretera oficial.

2. PLANTEAMIENTO DEL PROBLEMA

Dada una lista de 18 ciudades y las distancias entre cada par, encontrar el orden de visita que minimice la distancia del ciclo completo. Se desea comparar dos algoritmos de búsqueda local:

- **Hill Climbing (ascenso de colina):** Parte de una ruta al azar y solo acepta cambios que la mejoran.

Es rápido, pero puede quedar atrapado en un mínimo local.

- **Simulated Annealing (recocido simulado):** También puede aceptar cambios que empeoran la ruta, con una probabilidad que baja conforme se "enfría" el sistema. Así puede escapar de mínimos locales.

Preguntas a responder: ¿Cuál llega a mejores rutas?, ¿Qué tan cerca del óptimo real?, ¿Qué tan estables son los resultados entre corridas?, ¿Cómo influyen los parámetros T (temperatura), el número de iteraciones y n (enfriamiento)?

3. FORMULACIÓN

ELEMENTO	DEFINICÓN
Estado	Una permutación de las 18 ciudades (ruta cerrada). Ej.: [3, 7, 0, …, 12]
Estado inicial	Permutación aleatoria (semilla distinta por corrida). Costo medio inicial: 7,663 km.
Función objetivo	f(ruta) = Σ D[ruta i, ruta i+1] + D[rutan−1, ruta0] (km). Se minimiza
Vecino (movimiento 2-opt)	Se eligen dos posiciones i < j y se invierte el tramo entre ellas (Croes, 1958). Hay C (17,2) = 136 vecinos por estado. El cambio de costo Δ se calcula en O (1) comparando solo las 2 aristas que cambian.
Criterio de aceptación HC	Acepta el vecino solo si Δ < 0.
Criterio de aceptación SA	Acepta si Δ < 0; si no, con probabilidad exp(−Δ / T) (criterio de Metrópolis).
Parámetro T	Temperatura inicial T0 = 300 km (misma unidad que el costo).
Parámetro η (eta)	Factor de enfriamiento geométrico: T ← η·T en cada iteración. η = 0.9995. (En problemas

	continuos η suele ser el tamaño de paso; aquí, al ser discreto, se usa como razón de enfriamiento.)
Iteraciones	20,000 por corrida, igual para ambos algoritmos (comparación justa).
Repeticiones	30 corridas por algoritmo con semillas 1 a 30; ambos parten de la misma ruta inicial.

Con T_0 = 300 y η = 0.9995, la temperatura final es T0 ·η^{2000}0 ≈ 0.014 km, es decir, el sistema queda prácticamente "congelado" al final. Ejemplo de aceptación: con T = 300 un empeoramiento de 100 km se acepta con probabilidad $e^{-\frac{1}{3}}$ ≈ 0.72; con T = 10 baja a e^{-10} ≈ 0.00005

4. PROCEDIMIENTO

```
HILL CLIMBING
ruta <- permutacion aleatoria
para it = 1 .. N:
    (i,j) <- vecino 2-opt al azar
    delta <- cambio de costo
    si delta < 0:
        aplicar movimiento
    registrar costo
devolver ruta
```

```
SIMULATED ANNEALING
ruta <- permutacion aleatoria; T <- T0
mejor <- ruta
para it = 1 .. N:
    (i,j) <- vecino 2-opt al azar
    delta <- cambio de costo
    si delta < 0  o  azar() < exp(-delta/T):
        aplicar movimiento
        si costo < mejor: guardar mejor
    T <- eta * T
devolver mejor
```

Pasos seguidos:

- Se cargaron 18 ciudades y se construyó la matriz de distancias (haversine).
- Se programaron la función objetivo, el vecino 2-opt con cálculo incremental de Δ, Hill Climbing y Simulated Annealing.
- Se ejecutaron 30 corridas de cada algoritmo (20,000 iteraciones), guardando el historial de costos.
- Se calculó el óptimo exacto con Held-Karp como referencia.
- Se hizo un análisis de sensibilidad de SA (4 valores de T0 × 3 de η, 10 corridas cada uno) y del número de iteraciones.
- Se generaron las gráficas de convergencia, rutas, comparación y sensibilidad.
- Se incorporaron visualizaciones animadas de Hill Climbing y Simulated Annealing para observar los cambios en las rutas, la evolución del costo y el comportamiento de cada algoritmo durante el proceso de optimización.

5. RESULTADOS

Comparación de resultados (30 corridas, 20,000 iteraciones)

Métrica (km)	Hill Climbing	Simulated Annealing	Óptimo exacto
Mejor	3,617.6	3,617.6	3,617.6
Media	3,730.6	3,623.4	-
Mediana	3,688.0	3,617.6	-
Peor	4,171.3	3,642.4	-
Desviación estándar	121.9	10.7	-
Corridas que alcanzan el óptimo	7 de 30	23 de 30	-
Error medio vs óptimo	3.12 %	0.16 %	0 %
Tiempo total de 30 corridas	1.7 s	2.0 s	6.4 s (1 corrida)

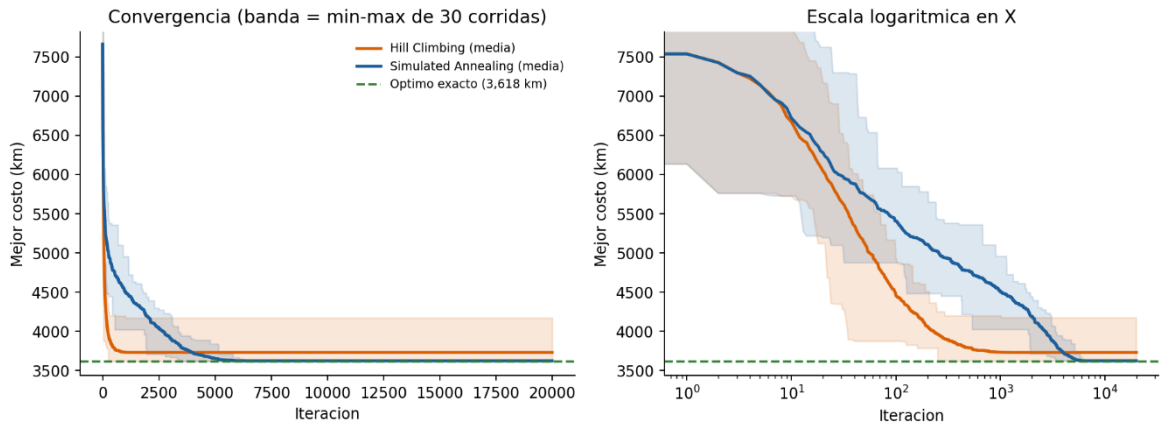


Figura 1. Curvas de convergencia (mejor costo vs. Iteración). Línea: media de 30 corridas; banda: mínimo- máximo.

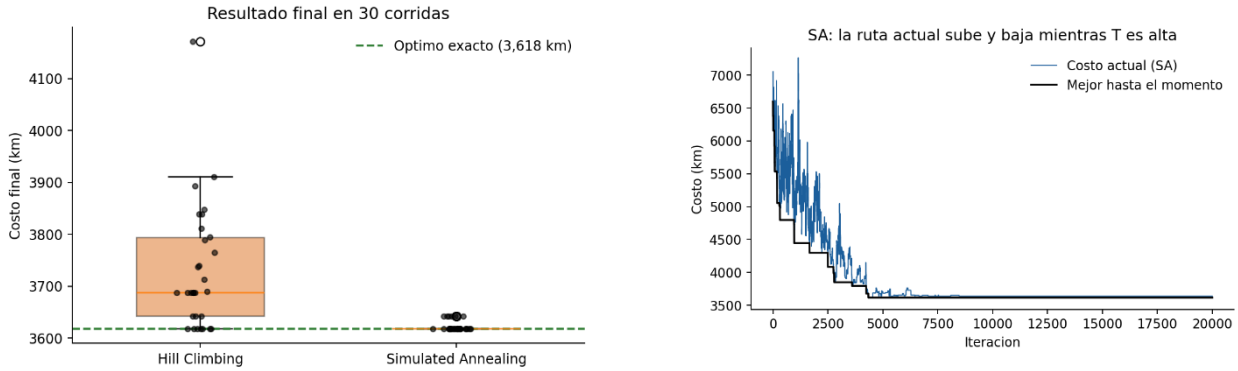


Figura 2 (izq.). Distribución del costo final en 30 corridas. Figura 3 (der.). SA: costo actual vs. mejor encontrado (una corrida)

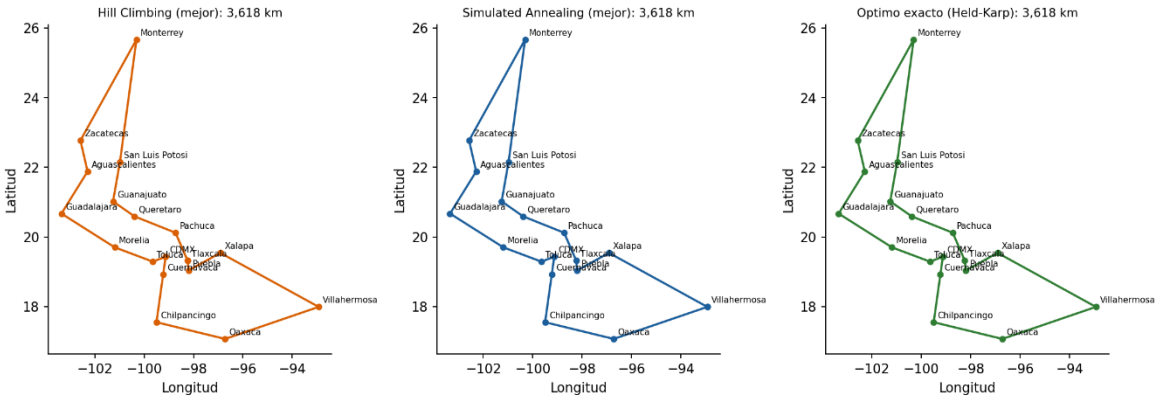


Figura 4. Mejor ruta hallada por cada método sobre el mapa (longitud/latitud) y el óptimo exacto.

Ruta óptima (Held-Karp): Ciudad de Mexico → Toluca → Morelia → Guadalajara → Aguascalientes → Zacatecas → Monterrey → San Luis Potosi → Guanajuato → Queretaro → Pachuca → Tlaxcala → Puebla → Xalapa → Villahermosa → Oaxaca → Chilpancingo → Cuernavaca → Ciudad de Mexico. Longitud: 3,617.63 km. Tanto HC (en sus mejores corridas) como SA encontraron una ruta de esta misma longitud.

Efecto de los parámetros T_0 y η en Simulated Annealing (10 corridas por celda)

T_0 (km)	η	Media (km)	Mejor (km)	Peor (km)	T final ≈ (km)
10	*0.999*	*3,655.0*	*3,617.6*	*3,755.1*	*2.04e-08*
10	*0.9995*	*3,648.3*	*3,617.6*	*3,688.0*	*0.000453*
10	*0.9999*	*3,632.1*	*3,617.6*	*3,712.8*	*1.35*
100	*0.999*	*3,632.1*	*3,617.6*	*3,712.8*	*2.04e-07*
100	*0.9995*	*3,626.2*	*3,617.6*	*3,678.9*	*0.00453*
100	*0.9999*	*3,617.6*	*3,617.6*	*3,617.6*	*13.5*

300	*0.999*	*3,627.6*	*3,617.6*	*3,642.4*	*6.12e-07*
300	*0.9995*	*3,625.1*	*3,617.6*	*3,642.4*	*0.0136*
300	*0.9999*	*3,630.4*	*3,617.6*	*3,688.0*	*40.6*
1000	*0.999*	*3,652.8*	*3,617.6*	*3,732.7*	*2.04e-06*
1000	*0.9995*	*3,636.5*	*3,617.6*	*3,688.0*	*0.0453*
1000	*0.9999*	*4,071.7*	*3,925.6*	*4,250.3*	*135*

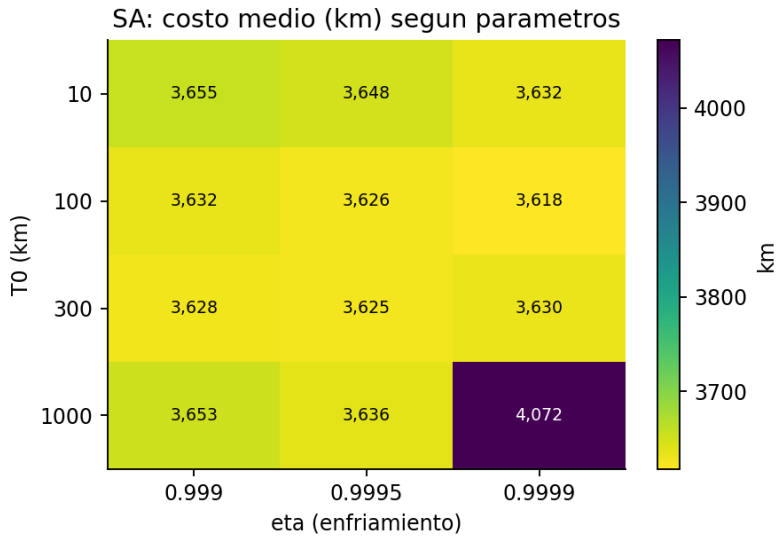


Figura 5. Costo medio de SA según T_0 y η (más claro/amarillo = peor).

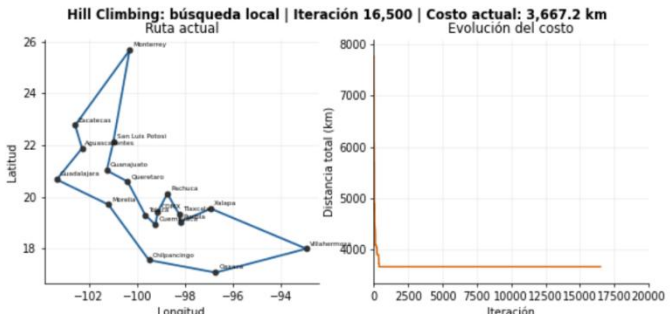
Efecto del número de iteraciones (10 corridas; el enfriamiento total se mantiene igual)

Iteraciones	HC: costo medio (km)	SA: costo medio (km)
1,000	3,731.4	3,751.5
5,000	3,731.4	3,646.6
20,000	3,731.4	3,625.1
50,000	3,731.4	3,617.6

Visualización animada de los algoritmos

Para complementar los resultados obtenidos, se generaron animaciones que permiten observar gráficamente la evolución de las rutas durante la búsqueda local. En Hill Climbing se muestran los cambios que reducen la distancia total, mientras que en Simulated Annealing se visualizan los movimientos que pueden aceptar temporalmente soluciones de mayor costo para explorar otras alternativas.

Estas animaciones facilitan la comprensión del proceso de optimización y complementan las gráficas estáticas, las tablas estadísticas y la comparación entre ambos algoritmos.



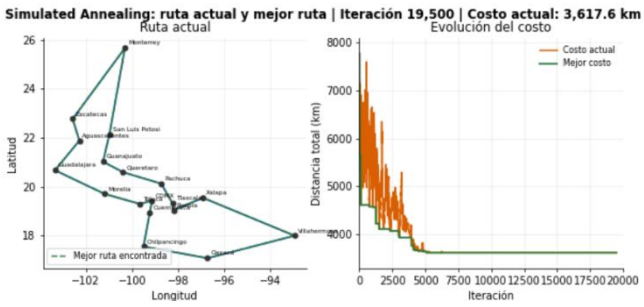


Figura 6. Animación del algoritmo Hill Climbing. Figura7. Animación del algoritmo Simulated Annealing. Elaboración propia.

6. ANÁLISIS

Análisis de resultados

Calidad: ambos reducen la ruta inicial de ≈ 7,663 km a menos de 3,731 km, pero SA queda en promedio a 0.16 % del óptimo y HC a 3.12 %. SA alcanzó el óptimo exacto en 23 de 30 corridas; HC solo en 7.

Estabilidad: la desviación estándar de HC (121.9 km) es unas 11 veces la de SA (10.7 km). El resultado de HC depende mucho de la ruta inicial; el peor caso de HC (4,171 km) es un mínimo local profundo.

Velocidad de convergencia: HC converge en alrededor de mil iteraciones y después la curva queda plana (atrapado). SA mejora más lento al inicio, porque acepta empeoramientos mientras T es alta, pero sigue mejorando y supera a HC alrededor de las 4,000–5,000 iteraciones.

Costo computacional: casi igual (≈ 2 s por 30 corridas en ambos). Cada iteración cuesta O(1) gracias al cálculo incremental de Δ. El método exacto tardó ≈ 6 s para 18 ciudades y crece como $O(n^2 \cdot 2n)$; con 30 ciudades sería inviable, mientras que HC y SA seguirían tomando segundos.

Iteraciones: con muy pocas (1,000) HC es mejor que SA, porque SA aún no se ha enfriado; con 5,000 o más SA gana, y con 50,000 llega al óptimo en las 10 corridas. HC no mejora al darle más iteraciones.

Parámetros: la mejor combinación fue T_0 = 100 y η = 0.9999 (óptimo en las 10 corridas). T_0 = 10 se parece a HC porque casi no acepta empeoramientos. La peor fue T_0 = 1000 con η = 0.9999: la temperatura final (≈ 135 km) sigue alta, el sistema no alcanza a “congelarse” en 20,000 iteraciones y termina en rutas malas (media ≈ 4,072 km). Es decir, T_0 y η deben ajustarse al número de iteraciones

7. CONCLUSIÓN

Hill Climbing es una buena primera aproximación: simple, rápido y suficiente cuando se acepta una solución “razonable”. Su debilidad es que depende del punto de partida y no puede salir de un mínimo local. Una forma sencilla de mejorarlo sería repetirlo con reinicios aleatorios. Simulated Annealing paga con un poco más de complejidad (dos parámetros que afinar) y obtiene soluciones más consistentes y casi óptimas; aprendí que el calendario de enfriamiento importa tanto como el algoritmo, pues un T0 alto con pocas iteraciones puede dar peores resultados que HC.

Limitaciones: las distancias son en línea recta con coordenadas aproximadas; el análisis de sensibilidad usó solo 10 corridas por combinación (y reutiliza semillas de las 30 principales), por lo que sus valores son indicativos; y el caso (18 ciudades) es pequeño. Para un uso real habría que usar distancias por carretera (por ejemplo, de una API de mapas) y probar con más ciudades. En aplicaciones de transporte, una mejora de pocos puntos porcentuales en la ruta se traduce en ahorro de combustible y tiempo, por lo que la calidad de la optimización tiene un impacto práctico.

8. REFERENCIAS

Anthropic. (2026). *Claude Sonnet 5.5* [Modelo de lenguaje de gran tamaño]. https://claude.ai

Croes, G. A. (1958). A method for solving traveling-salesman problems. *Operations Research, 6*(6), 791–812. https://doi.org/10.1287/opre.6.6.791

García Serrano, A. (2016). *Inteligencia artificial: Fundamentos, práctica y aplicaciones* (2.ª ed.). RC Libros.

Harris, C. R., Millman, K. J., van der Walt, S. J., Gommers, R., Virtanen, P., Cournapeau, D., Wieser, E., Taylor, J., Berg, S., Smith, N. J., Kern, R., Picus, M., Hoyer, S., van Kerkwijk, M. H., Brett, M., Haldane, A., del Río, J. F., Wiebe, M., Peterson, P., ... Oliphant, T. E. (2020). Array programming with NumPy. *Nature, 585*, 357–362. https://doi.org/10.1038/s41586-020-2649-2

Held, M., & Karp, R. M. (1962). A dynamic programming approach to sequencing problems. *Journal of the Society for Industrial and Applied Mathematics, 10*(1), 196–210. https://doi.org/10.1137/0110015

Hunter, J. D. (2007). Matplotlib: A 2D graphics environment. *Computing in Science & Engineering, 9*(3), 90–95. https://doi.org/10.1109/MCSE.2007.55

Kirkpatrick, S., Gelatt, C. D., & Vecchi, M. P. (1983). Optimization by simulated annealing. *Science, 220*(4598), 671–680. https://doi.org/10.1126/science.220.4598.671

Python Software Foundation. (s. f.). *Python*. https://www.python.org/

Russell, S., & Norvig, P. (2020). *Artificial intelligence: A modern approach* (4th ed.). Pearson.

Sinnott, R. W. (1984). Virtues of the haversine. *Sky & Telescope, 68*(2), 159.

Wirsansky, E. (2020). *Hands-on genetic algorithms with Python: Applying genetic algorithms to solve real-world deep learning and artificial intelligence problems*. Packt Publishing.

9. DECLARACIÓN DE USO DE INTELIGENCIA ARTIFICIAL

Herramienta: Claude Sonnet 5.5 (Anthropic), usado mediante Claude.ai.

Prompt resumido: Implementa Hill Climbing y Simulated Annealing para optimizar una función real, dame información de: enunciado, función objetivo, vecindad, parámetros (T, iteraciones, η)

Uso: Apoyo en la formalización de los problemas y generación del código base

Verificación: El código fue ejecutado localmente, los grafos se generaron correctamente, los diagramas se revisaron visualmente y el análisis de complejidad se contrastó con la teoría de Russell & Norvig.